

Mico-UI自动化测试全流程

一、背景

IPD 米立方原子化研发-测试流程中，每个优化迭代都会产出大量 Web 页面——新建页、详情页、列表页回归验证等。这些页面的功能质量直接影响测试交付效率，而 **UI 功能测试** 是保障页面质量的关键手段。

然而，传统的 UI 测试面临几个核心挑战：

- 人力密集**：每轮迭代需人工逐页面、逐字段验证，回归测试耗时巨大
- 资产浪费**：功能测试做完即丢，无法沉淀为可复用的回归资产
- 自动化缺失**：人工手写自动化脚本依赖页面元素定位（XPath/CSS），耗时效率低

二、现状痛点与优化方向

**传统手工测试**

- 用例靠人手工执行，测试卡点
- 失败用例人工复现，手工截图和保留错误日志
- 每轮用例测试无法沉淀为可复用自动化资产
- 依赖测试人工判定测试结果，无二次复核验证
- 测试结束手动填写产出报告

**Mico UI自动化测试**

- Agent自动解析用例、自动执行
- trace/video 录制，失败可逐帧回放
- 通过用例**自动生成UI自动化脚本沉淀进仓库
- 审核者独立复核测试结果二次验证
- 产出测试报告、缺陷提交与提交自动化MR

三、设计方案与架构

3.1 设计目标

**一句话目标**：让 AI 代替人工完成「页面探索 → 自动执行 → 代码生成 → 资产沉淀」全链路，实现功能测试与 UI 自动化的闭环。

3.2 设计方案：CLI 探路 + Test 回归

	Playwright CLI	Playwright Test
是什么	命令行工具,一条条操作浏览器	JavaScript/TypeScript测试框架
怎么用	playwright-cli click e12	await page.getByRole('button').click()
写在哪	Shell命令/JSON计划	spec.ts测试文件
典型场景	探索页面、人工辅助、快速验证	CI自动化、长期回归测试

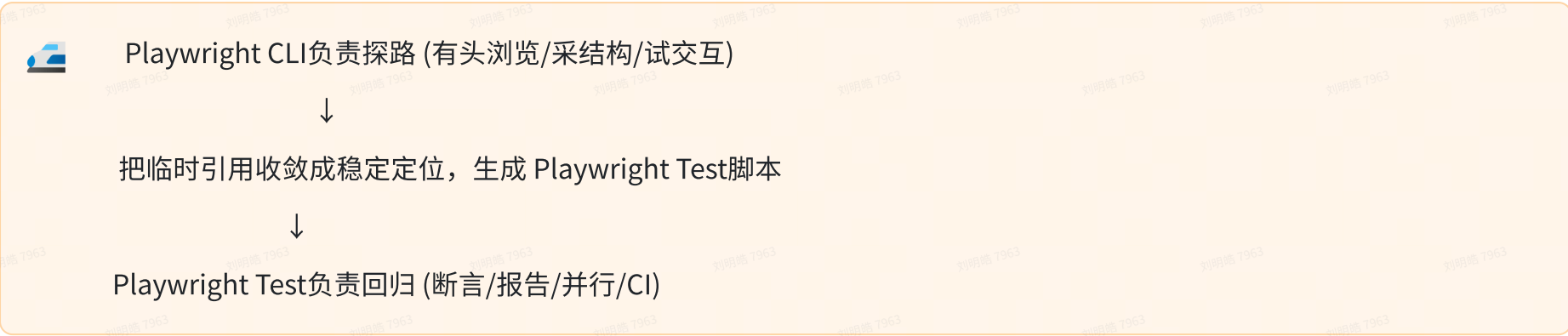
概念澄清

1. **Playwright Test**：Playwright官方主推的端到端测试框架。

相对「随便写一段自动化脚本」,它补齐了测试工程侧能力:用例组织、断言、Fixture、自动等待、并行、报告等。

2. **Playwright CLI**：面向命令行/Agent的浏览器操控入口：打开页面、看当前结构、点选填充、截图录屏等，都可以用轻量指令完成。

实现思路



- ✅探索阶段保留CLI的速度:不必每改一步就改spec、整包重跑
- ✅回归阶段切到Playwright Test:脚本成为可版本管理的工程资产
- ✅临时引用不再直接进回归:探索结束后转为更稳妥的定位写法
- ✅成功率更贴近常态化执行:适合重复跑

四、实现流程

4.1 Mico 协作架构

测试小队采用 **4 Agent 五角色** 模式：

角色	职责
队长	任务拆解分派、强制约束前置、产物核对、审核分派、最终结论
测试探索执行者	专注 Playwright Test 执行功能测试，产出测试报告+录制回放+har文件（API 小队使用）
UI自动化生成者	依赖测试探索产物生成UI自动化测试代码，mvn test验证后提交MR（不做独立探索）
审核者	独立复测、终审复核、给出通过/不通过结论

设计决策：

- **强制约束前置**：队长分派时即明确约束（如"以 Playwright 原生报告为唯一事实来源"），避免执行者自行发挥导致返工
- **独立审核**：审核者独立解码 Playwright Test测试产物，不依赖执行者的文字描述
- **闭环升级**：用例测试通过 → 自动触发UI自动化生成，验证与沉淀一体化，同一套用例只需"跑一遍"

2道门禁评审规则：

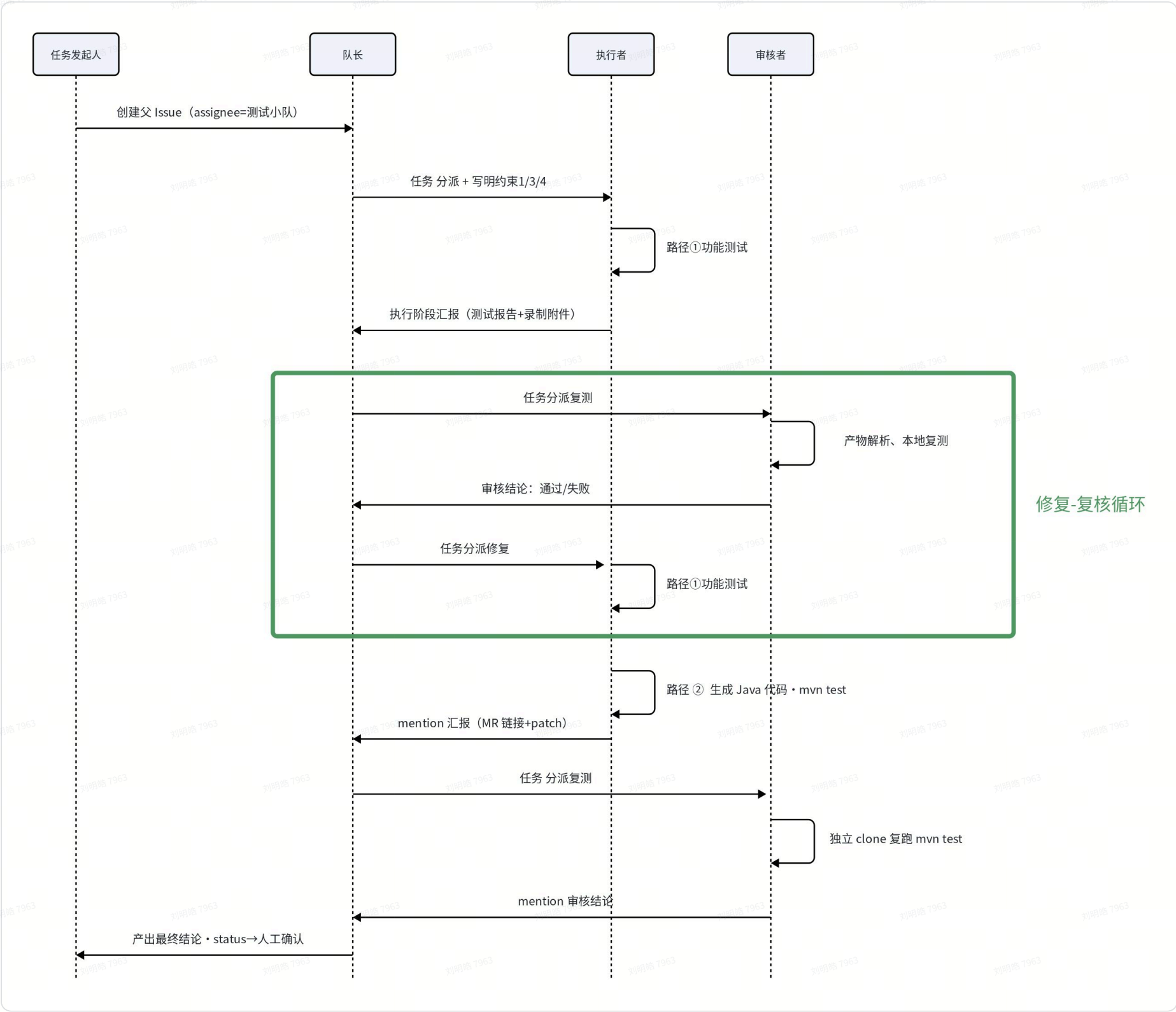
门禁	时机	评审内容	打回动作
门禁1：AI*测试	AI*测试执行者完成后	测试结果是否可信、失败用例是否有根因、报告+录制产物是否完整	打回执行者补测/修复

门禁2：UI 自动化	UI自动化生成者完成后	代码是否基于AI*产物生成、mvn test是否通过、MR信息是否正确	打回生成者修复代码
------------	-------------	-------------------------------------	-----------

测试小队执行路径分层架构

路径	触发条件	技术栈	产物归属	用途
路径1：AI*功能测试	"执行用例测试"	Playwright Test（JS/TS）	MICO上传	快速功能验证
路径2：UI 自动化生成	"生成 UI 自动化测试代码"	Java + Playwright Java + TestNG + metal	进自动化测试仓库，走 MR	长期回归、CI 门禁

协作流程图



完整 workflow:

```
代码块
用例来源 (teego导入/mico用例目录)
2      ↓
3  ① 测试探索执行者 (功能测试探索)
4      ↓
5  ② 评审者 (门禁1: AI*测试审批)
6      ↓
7  ③ UI自动化生成者 (严格依赖①的产物生成)
8      ↓
9  ④ 评审者 (门禁2: UI自动化审批)
10     ↓
11 ⑤ API测试小队 (传递.har文件)
12
```

```
代码块
1  创建 Issue
2  → 队长分派 测试探索执行者 → 执行者功能测试 → 评审者门禁2审批
3  → 队长分派UI自动化生成者 → 生成者代码沉淀 → 评审者门禁3审批
4  → 队长汇总结论 → Issue 状态流转
```

首轮：队长拆解任务 → 执行者执行 → 向队长汇报 → 队长核对完整性 → 审核者独立复核 → 队长汇总结论

返工：审核者复核失败，提返工要求 → 队长细化分派 → 执行者修改重上传 → 队长核对 → 审核者审核 → 闭环

4.2 路径1：AI*功能测试

```
🚀 解析用例 → Cookie 注入 → 页面探索 → 生成 Page Object → 生成测试代码 → 代码校验 → 自动执行 → 测试报告 → 静态报告 → 上传产物 → 升级路径2
```

```
代码块
1  注意点：UI测试适用于单系统页面闭环、不涉及到上下游链路驱动的功能测试
```

Step 1：解析输入

接收并解析测试用例文件，可以为Teego测试计划链接

以 Excel 为例，每个 Sheet 对应一个功能模块，列含用例名称/前置条件/步骤/预期/优先级。

- Mico测试用例：自动化巡检-流程引擎流转用例
- 📄 新建页详情页核心场景用例.xlsx

Step 2：CAS Cookie 注入

```
代码块
1  系统名查映射表 → 调 API 获取 Cookie → 注入 Chrome → 自动登录
```

Step 3：页面探索（Playwright CLI）

有头模式下打开目标测试页面，采集页面快照和元素结构，产出 console.log 和 page_structure.yml 文件。

如有图形验证码，使用大模型内置 read 功能识别。

`page-locators_流程实例__流程流转_新建产品需求实例依次完成各节点审批至流程结束.yml`

Step 4：生成 Page Object

测试用例AI探索对应目标页面快照、DOM：按

Locator

 优先级 生成稳定的 Page Object：

代码块

- 1
- getByTestId() – 项目约定的 data-testid，最优先
- 2
- getByRole() + getByText() – 语义化、稳定
- 3
- getByPlaceholder() / getByLabel() – 表单元素
- 4
- css selector – 兜底手段，需注释说明

Step 5：生成测试代码

根据 Page Object 和用例执行步骤，生成 spec.ts 测试文件，每条用例自包含（beforeEach 重置状态）。

Step 6：代码校验 + 自动执行测试用例

ESLint + tsc 检查，自动修复后执行 Playwright Test（默认开启 trace + video 录制）。

Step 7：测试报告产物

- 测试报告汇总测试结果，支持可交互查看每步操作
- 输出测试用例执行录制回放，支持人工复核验证



新建页心场景

【必填校验】新建工作项实例-必填字段未填写-提交失败

new-page.spec.ts:36

33.7s

chromium

Run

Errors

Error: 应有必填校验提示, toast='树形视图-自关联工作项创建成功', nameHasError=false

expect(received).toBeTruthy()

Received: false

51 | return /error|invalid|required/.test(cls + parentCls);

52 | }, catch() => false);

53 | expect(hasRequiredTip || nameHasError, `应有必填校验提示, toast='\${toast}', nameHasError=\${nameHasError}`).toBeTruthy();

54 | });

55 |

56 | test('【保存草稿】新建工作项实例-仅填写标题-存草稿, 保存成功', async ({ page }) => {

at C:\Users\lai\workspace\bbdb7853-7507-4285-915f-ad9801d8f8d5\workdir\path2\tests\new-page.spec.ts:53:103

Test Steps

- > ✓ Before Hooks8.6s
- > ✓ locator.getByRole('button', { name: '新建', exact: true }).first().click — workitem-new.po.ts:5973ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:60833ms
- > ✓ locator.click([].hi-v5-dropdown-menu-item >> internal:has-text='IR需求') >> nth=0 — workitem-new.po.ts:65127ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:662.5s
- > ✓ expect.toBeVisible — workitem-new.po.ts:6719ms
- > ✓ expect.toBeVisible — new-page.spec.ts:3816ms
- > ✓ locator.click([role='dialog'] >> nth=0 >> internal:role=button[name='提交's]) — workitem-new.po.ts:85133ms
- > ✓ expect.toBeVisible — new-page.spec.ts:4225ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11235ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115219ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11215ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115213ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11215ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115222ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11217ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115216ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11216ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115216ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11216ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115220ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11224ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115228ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11223ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115215ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11222ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115216ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11228ms
- > ✓ page.waitForTimeout — workitem-new.po.ts:115273ms
- > ✓ locator.allInnerTexts([class='hi-v5-message']) — workitem-new.po.ts:11228ms
- > ✓ locator.evaluate([role='dialog'] >> nth=0 >> input[name='name']) — new-page.spec.ts:4715.1s
- > ✗ 应有必填校验提示, toast='树形视图-自关联工作项创建成功', nameHasError=false — new-page.spec.ts:532ms
- > ✓ After Hooks3.8s
- > ✓ Worker Cleanup427ms

Screenshots

Step 8：生成UI自动化

测试用例通过后，自动升级到路径2：UI自动化生成

4.3 路径2：UI自动化生成

拉取仓库 → 读取代码配置 → 解析Playwright Test产物 → 生成 Java 代码 → 本地 mvn test 通过 → 提交 MR

关键复用：生成代码不是从零开始，而是复用仓库现有基础设施：

现成能力	位置/类	复用方式
测试基类	IPDBaseTest	生成的测试类继承它
CAS 登录态	CasBrowserContext	复用，不自己造 storageState
录制回放	@RecordVideo + ScreenRecordingManager	加注解即开启

UI 失败监听	UIFailureListener	基类已注册
用例标注	@UITest / @Owner	按现有范式加注解
PO 模式	IssueListPo 等	生成的 PO 放同目录，同风格

提交MR 门禁：只有 mvn test 全部通过才可提交。

4.3 Mico 协作前后对比



改造前

- 回归靠人，周期以天计
- 结果口径主观、易失真
- 失败难复现、难归因
- 用例无法沉淀复用
- 无审核，质量单点依赖



改造后

- Agent自动执行，周期以小时计
- 原生报告客观、可交叉核对
- trace/video 逐帧回放
- 通过用例自动沉淀 Java 版
- 执行/审核分离，双闸放行

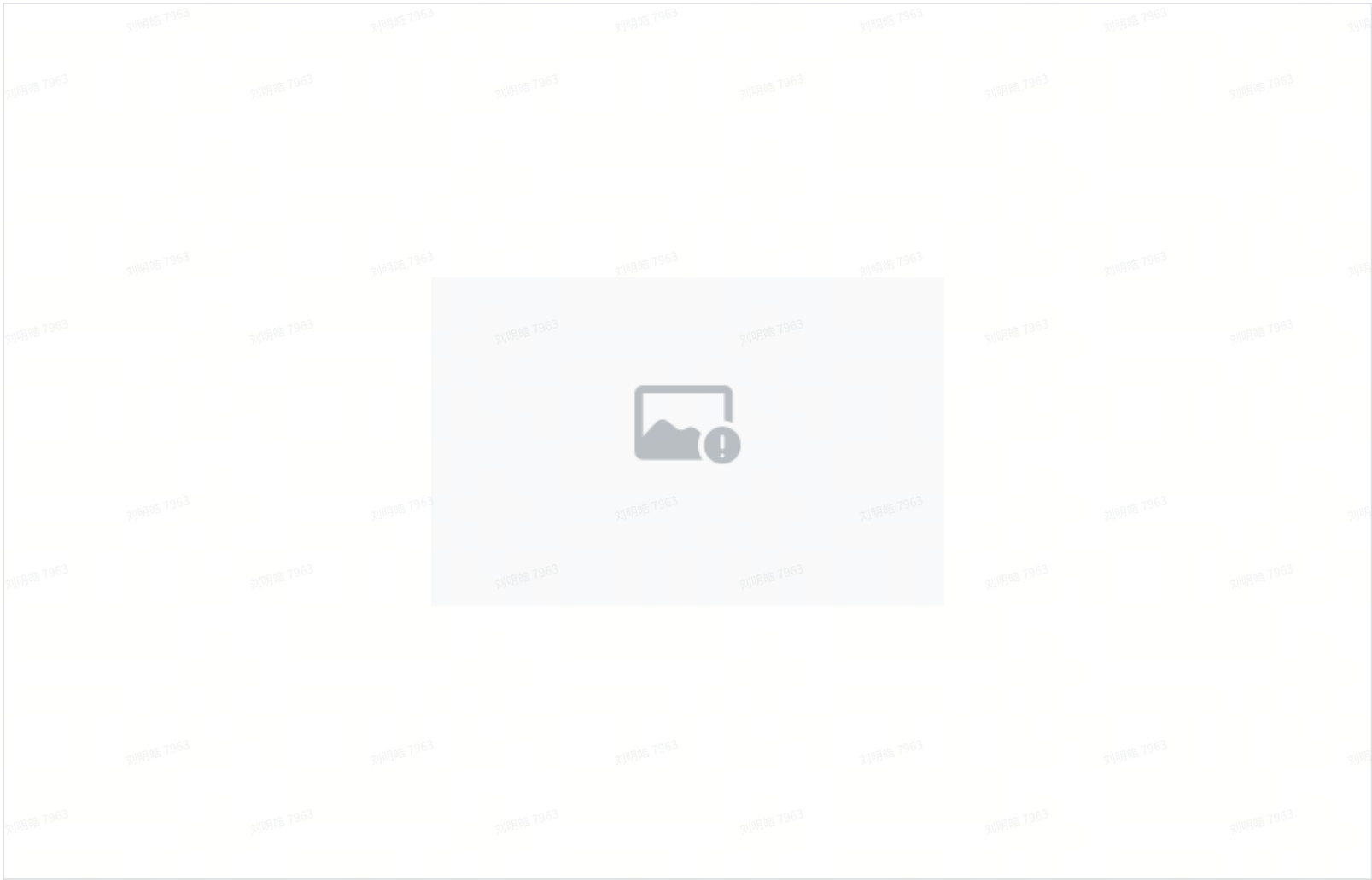
价值

- **提效**：用例执行+代码沉淀一条龙
- **保质**：原生报告为事实来源 + 独立复测，杜绝报告失真
- **沉淀**：通过即沉淀，自动化资产随版本持续增长
- **闭环**：失败 Case 自动分析定位，区分自动化缺陷与研发缺陷

五、Mico执行案例

1. 单条复杂审批流程链路用例、多条用例批量生成

- a. 输入测试用例：MiCo · 测试用例
- b. AI执行结果回放：



全文评论



刘明皓 8月18日 14:21

我想问下，老师你们，step3 step4 是具体怎么做的呀，console.log 和 page_structure.yml 里面是什么。我们之前有用 agent-browser，但是 ai 对网页结构的理解还是有问题，token 消耗量也很大



徐忠发 8月18日 19:08

@刘明皓 console.log 是 playwright 控制台的日志数据，另一个是 AI 在页面探索执行的每一步操作的元素定位器，就是把 AI 在执行时的 DOM 元素进行保留给到下一步生成 UI 自动化测试代码为来源



刘明皓 8月18日 20:35

@徐忠发 page_structure.yml 里面，操作对应的元素定位器，是人工介入 提前写好放在 page_structure.yml 里面的吗？这一步是不是工作量非常大



徐忠发 8月18日 20:52

@刘明皓 不是的，是 AI 自主在页面探索执行通过稳定，结合操作 trace 和 DOM 生成的



刘明皓 8月19日 12:01 （编辑过）

@徐忠发 感谢徐老师耐心解答。我差不多还有最后一个问题。你们 DOM 生成是不是用的 playwright 自带的 page.aria_snapshot()函数，它负责"让 AI 看懂页面"。你们的 console.log 是通过 page.on("console", ...) 这种事件监听方式，把浏览器 console 输出写成纯文本文件的对吗？不是 context.tracing.start() 产出的 trace.zip 吧？两者配合来判断一个 selector 能不能保存进 yml(点没点中、有没有报错)。这样理解对吗？



刘明皓 8月21日 10:21

@徐忠发 徐老师，你们的 excel 样例质量很高，是怎么生成的哦，人工介入多吗？例如下面样例中的「」半结构化描述：「新建」按钮，在下拉菜单中点击「产品需求」



徐忠发 8月21日 10:35

这个是让 AI 结合页面生成的，目前是手动给 AI 提示词来进行优化用例，后面会做成 Agent 融入到小队中